

✓ Dénormalisation avec PostgreSQL :

Vues, Tableaux, Types composites

✓ 1. Vues

1.1 Connaître la moyenne de chacune des évaluations est une information utile pour les commissions de passage ainsi que pour l'édition des bulletins. On souhaite que cette moyenne soit normalisée sur 20. Proposer une requête SQL qui affiche la table evaluation avec comme colonne supplémentaire la moyenne de l'évaluation sur 20.

```
SELECT id_evaluation, nom, dateeval, coefficient, echelle, id_module,  
AVG( note) / echelle * 20 AS moyenne_sur_20  
FROM evaluation JOIN notation USING (id_evaluation)  
GROUP BY id_evaluation, nom, dateeval, coefficient, echelle, id_module ;
```

1.2 Cette moyenne est si souvent utile que nous voudrions y avoir accès facilement sans avoir à effectuer de requête. Peut-on ajouter cette colonne à la table evaluation en utilisant une colonne générée ?

Réponse : Non, c'est impossible car l'information qui permet de générer cette colonne n'est pas exclusivement contenue dans la table evaluation mais répartie sur les tables evaluation et notation.

1.3 Créer une vue "vue_evaluation" avec cette colonne supplémentaire.

```
CREATE VIEW vue_evaluation AS  
SELECT id_evaluation, nom, dateeval, coefficient, echelle, id_module,  
AVG( note) / echelle * 20 AS moyenne_sur_20  
FROM evaluation JOIN notation USING (id_evaluation)  
GROUP BY id_evaluation, nom, dateeval, coefficient, echelle, id_module ;
```

1.4 Interprétez le plan d'exécution de la commande affichant le contenu de cette vue :

```
EXPLAIN ANALYSE SELECT * FROM vue_evaluation ;
```

Commencez à coder ou à [générer](#) avec l'IA.

```
=> EXPLAIN ANALYSE SELECT * FROM vue_evaluation ;
```

QUERY PLAN

```
-----...
HashAggregate  (cost=143.76..144.79 rows=59 width=54)
    (actual time=6.900..6.918 rows=59 loops=1)
    Group Key: evaluation.id_evaluation
    -> Hash Join  (cost=2.33..113.32 rows=6087 width=54)
        (actual time=0.074..4.573 rows=6087 loops=1)
        Hash Cond: (notation.id_evaluation = evaluation.id_evaluation)
        -> Seq Scan on notation  (cost=0.00..93.87 rows=6087 width=12)
            (actual time=0.021..1.144 rows=6087 loops=1)
        -> Hash  (cost=1.59..1.59 rows=59 width=46)
            (actual time=0.040..0.040 rows=59 loops=1)
            Buckets: 1024  Batches: 1  Memory Usage: 13kB
            -> Seq Scan on evaluation  (cost=0.00..1.59 rows=59 width=46)
                (actual time=0.009..0.018 rows=59 loops=1)

Planning Time: 0.447 ms
Execution Time: 7.009 ms
(10 rows)
```

Ce plan d'exécution montre une opération d'agrégat par hachage réalisée sur le résultat d'une jointure par hachage. Ce plan passe par une lecture séquentielle des tables evaluation et notation. Le temps total d'exécution est de 7 millisecondes.

1.5 Créer une vue matérialisée "vuemat_evaluation" avec cette colonne supplémentaire.

```
CREATE MATERIALIZED VIEW vuemat_evaluation AS
SELECT id_evaluation, nom, dateeval, coefficient, echelle, id_module,
AVG( note) / echelle * 20 AS moyenne_sur_20
FROM evaluation JOIN notation USING (id_evaluation)
GROUP BY id_evaluation, nom, dateeval, coefficient, echelle, id_module ;
```

1.6 Interprétez le plan d'exécution de la commande affichant le contenu de cette vue matérialisée :

```
EXPLAIN ANALYSE SELECT * FROM vuemat_evaluation ;
```

La commande est-elle plus rapide que sur la vue classique ? D'un facteur de combien ?

Commencez à coder ou à [générer](#) avec l'IA.

```
=> EXPLAIN ANALYSE SELECT * FROM vuemat_evaluation ;  
                                QUERY PLAN  
-----  
Seq Scan on vuemat_evaluation (cost=0.00..1.59 rows=59 width=54)  
                                (actual time=0.007..0.012 rows=59 loops=1)  
  
Planning Time: 0.127 ms  
Execution Time: 0.027 ms  
(3 rows)
```

La vue est matérialisée, la table est donc réellement remplie avec le résultat de la requête. La requête n'a donc pas besoin d'être exécutée de nouveau : le SGBD se contente de réaliser une lecture séquentielle de la table vuemat_evaluation. L'opération est bien plus rapide et prend 0, 027 millisecondes. Elle est donc 260 fois plus rapide.

1.7 Intéressons-nous à la requête affichant les informations concernant les évaluations du module 2 de la vue vue_evaluation :

```
SELECT * FROM vue_evaluation WHERE id_module = 2 ;
```

Le SGBD doit-il recalculer entièrement la vue vue_evaluation pour répondre à cette requête ? Cette requête est-elle plus lente ou plus rapide que celle exécutée plus haut question 4 consistant à afficher tout le contenu de la vue. Comment observer ce que le SGBD fait réellement ?

Réponse : Le problème de la vue est géré par le module Rewriter du compilateur de requête. Le système de réécriture remplace dans l'arbre de requête la référence à la vue par son sous-arbre correspondant, puis évalue le nouvel arbre qui ne contient plus que des tables de base. La stratégie d'accès aux données à partir de l'arbre de requête est définie par module de planification/optimisation et ne passera pas nécessairement par un recalcul complet de la vue. Pour savoir ce que le SGBD fait, il faut afficher le plan d'exécution de la requête :

Commencez à coder ou à [générer](#) avec l'IA.

```
=> EXPLAIN ANALYSE SELECT * FROM vue_evaluation WHERE id_module = 2 ;
                                QUERY PLAN
-----...
HashAggregate  (cost=113.79..113.82 rows=2 width=54)
                (actual time=3.311..3.315 rows=2 loops=1)
  Group Key: evaluation.id_evaluation
    -> Hash Join  (cost=1.76..112.76 rows=206 width=54)
              (actual time=0.126..3.157 rows=251 loops=1)
    Hash Cond: (notation.id_evaluation = evaluation.id_evaluation)
      -> Seq Scan on notation  (cost=0.00..93.87 rows=6087 width=12)
            (actual time=0.023..1.462 rows=6087 loops=1)
      -> Hash  (cost=1.74..1.74 rows=2 width=46)
            (actual time=0.032..0.033 rows=2 loops=1)
        Buckets: 1024  Batches: 1  Memory Usage: 9kB
        -> Seq Scan on evaluation  (cost=0.00..1.74 rows=2 width=46)
              (actual time=0.023..0.028 rows=2 loops=1)
          Filter: (id_module = 2)
          Rows Removed by Filter: 57
Planning Time: 0.476 ms
Execution Time: 3.394 ms
(12 rows)
```

Ce plan d'exécution nous permet de voir que l'opération de sélection `id_module = 2` est attachée au tout premier nœuds du plan : -> Seq Scan on evaluation. Seule la partie utile de la vue est donc reconstituée. Grâce à cela, l'exécution de cette requête sur la vue est plus rapide que la requête consistant à afficher toute la vue : 3, 4 millisecondes au lieu des 7 millisecondes.

1.8 Que dire de cette même requête sur la vue matérialisée :

```
SELECT * FROM vuemat_evaluation WHERE id_module = 2 ;
```

Affichons le plan d'exécution de la requête :

Commencez à coder ou à [générer](#) avec l'IA.

```
EXPLAIN ANALYSE SELECT * FROM vuemat_evaluation WHERE id_module = 2 ;
                                QUERY PLAN
-----...
Seq Scan on vuemat_evaluation  (cost=0.00..1.74 rows=2 width=54)
                                (actual time=0.018..0.034 rows=2 loops=1)
   Filter: (id_module = 2)
   Rows Removed by Filter: 57
Planning Time: 0.085 ms
Execution Time: 0.056 ms
(5 rows)
```

Ici le travail est le même que pour afficher toute la vue car il faut un parcours séquentiel complet de la table vuemat_evaluation pour appliquer l'opération de sélection id_module = 2. Donc aucun gain de temps par rapport à la requête de la question 6. Mais la requête reste bien plus rapide que celle sur la vue (0, 056 millisecondes au lieu des 3, 4 millisecondes).

2. Tableaux

Écrire une requête qui donne le nombre d'évaluations de chacun des modules.

```
SELECT id_module, intitule, COUNT(*) AS "Nb. eval."
FROM module JOIN evaluation USING(id_module)
GROUP BY id_module, intitule ;
```

En partant de la requête précédente, écrire une requête qui donne, sous forme de tableau, la liste des identifiants des évaluations de chacun des modules. La fonction d'agrégation array_agg() est parfaitement adaptée à cette situation.

```
SELECT id_module, intitule, array_agg(id_evaluation) AS "Tab. eval."
FROM module JOIN evaluation USING(id_module)
GROUP BY id_module, intitule ;
```

Modifier la requête précédente pour avoir, non plus un tableau d'identifiants, mais un tableau de couples (identifiant,nom) (i.e. un tableau à deux dimensions). Comme un tableau ne contient que des éléments d'un même type, il faudra transtyper les identifiants en chaîne de caractères.

```
SELECT id_module, intitule,
array_agg(ARRAY[id_evaluation::varchar, nom]) AS "Tab. eval."
```

```
FROM module JOIN evaluation USING(id_module)
GROUP BY id_module, intitule ;
```

Écrire une requête qui affiche pour chaque évaluation les notes maximum et minimum sur 20.

```
SELECT id_evaluation, nom, id_module,
ROUND(MAX(note/echelle*20)::numeric,1) AS "max",
ROUND(MIN(note/echelle*20)::numeric,1) AS "min"
FROM evaluation JOIN notation USING(id_evaluation)
GROUP BY id_evaluation, nom, id_module ;
```

Réécrire la requête de la question 12, en utilisant la requête de la question 11 en tant que CTE, pour ajouter les colonnes des notes maximum et minimum à chacune des évaluations.

```
WITH eval AS (
SELECT id_evaluation, nom, id_module,
ROUND(MAX(note/echelle*20)::numeric,1) AS "max",
ROUND(MIN(note/echelle*20)::numeric,1) AS "min"
FROM evaluation JOIN notation USING(id_evaluation)
GROUP BY id_evaluation, nom, id_module
)
SELECT id_module, intitule, array_agg(ARRAY[id_evaluation::varchar,
nom, max::varchar, min::varchar])
FROM module JOIN eval USING(id_module)
GROUP BY id_module, intitule ;
```

Faire de la requête précédente une vue. Penser à nommer la dernière colonne. Les notes doivent être arrondies à un chiffre après la virgule (utiliser la fonction ROUND(::numeric,1)).

```
CREATE VIEW vue_module_tab AS
WITH eval AS (
SELECT id_evaluation, nom, id_module,
ROUND(MAX(note/echelle*20)::numeric,1) AS "max",
ROUND(MIN(note/echelle*20)::numeric,1) AS "min"
FROM evaluation JOIN notation USING(id_evaluation)
GROUP BY id_evaluation, nom, id_module
)
SELECT id_module, intitule, array_agg(ARRAY[id_evaluation::varchar,
nom, max::varchar, min::varchar]) AS evaluations
FROM module JOIN eval USING(id_module)
GROUP BY id_module, intitule ;
```

En utilisant cette vue, écrire une requête qui affiche pour chaque module le nom de la première évaluation et sa note maximale.

```
SELECT intitule, evaluations[1][2] AS nom, evaluations[1][3] AS max
FROM vue_module_tab ;
```

3. Types composites (ou UDT pour User Defined Types)

Créer un type composite pour représenter un identifiant, un nom, une note maximum et une note minimum.

```
CREATE TYPE info_eval AS (
  id_evaluation INTEGER,
  nom VARCHAR,
  max REAL,
  min REAL
) ;
```

Supprimer la vue de la question 14.

```
DROP VIEW vue_module_tab ;
```

Recréer une vue équivalente à celle de la question 14 avec des tableaux unidimensionnels constitués d'éléments du type déclaré dans la question 16.

```
CREATE VIEW vue_module_tab AS
WITH eval AS (
  SELECT id_evaluation, nom, id_module,
  ROUND(MAX(note/echelle*20)::numeric,1) AS "max",
  ROUND(MIN(note/echelle*20)::numeric,1) AS "min"
  FROM evaluation JOIN notation USING(id_evaluation)
  GROUP BY id_evaluation, nom, id_module
)
SELECT id_module, intitule,
array_agg((id_evaluation, nom, max, min)::info_eval) AS evaluations
FROM module JOIN eval USING(id_module)
GROUP BY id_module, intitule ;
```

En utilisant cette vue, écrire une requête qui affiche pour chaque module le nom de la première évaluation et sa note maximale.

```
SELECT intitule, evaluations[1].nom, evaluations[1].max  
FROM vue_module_tab ;
```